

Khung Toán Học và Thuật Toán Tối Ưu Trong MLOps: Từ Khái Niệm, Lập Lý Thuyết Đến Phân Tích Mã Giả Định

Phần Mở Đầu: Sự Chuyển Dịch Sang Hệ Sinh Thái MLOps 2.0 Định Lượng

Sự phát triển theo cấp số nhân của quy mô, độ phức tạp và tính đa dạng trong các hệ thống học máy (Machine Learning - ML), đặc biệt là sự trỗi dậy của các Mô hình Ngôn ngữ Lớn (LLMs) và các hệ thống chẩn đoán y tế tự động, đã và đang đặt ra những thách thức chưa từng có đối với việc triển khai và quản lý vận hành trong môi trường thực tế.¹ MLOps (Machine Learning Operations), ban đầu chỉ được hiểu một cách đơn giản là việc áp dụng các nguyên tắc DevOps cho quy trình học máy, nay đã tiến hóa mạnh mẽ thành một hệ sinh thái phức tạp thường được giới nghiên cứu gọi là MLOps 2.0. Trong kỷ nguyên mới này, sự quản trị (governance), khả năng kiểm toán (auditability), tính minh bạch và sự bền bỉ của hệ thống không còn dựa trên các quy trình thủ công hay kịch bản (scripts) định tuyến tĩnh, mà phải được định lượng một cách khắt khe thông qua các khuôn khổ toán học chặt chẽ.³

Trọng tâm cốt lõi của tối ưu hóa trong MLOps hiện đại là cung cấp một khung toán học để lựa chọn các phương án tốt nhất từ một tập hợp vô cùng lớn các giải pháp thay thế, dưới hàng loạt các ràng buộc khắt khe về chi phí, tài nguyên, thời gian tính toán và độ chính xác.⁵ Sự chuyển dịch mô hình (paradigm shift) này đòi hỏi hệ thống phải tự động đưa ra các quyết định liên quan đến cấp phát tài nguyên đa luồng, giám sát trôi dạt dữ liệu (data drift), tinh chỉnh siêu tham số và tối ưu hóa quy trình bảo trì dựa trên việc giải quyết các bài toán tối ưu hóa, quy hoạch nguyên (integer programming) và các quá trình quyết định Markov (Markov Decision Processes - MDP).⁵

Báo cáo nghiên cứu chuyên sâu này trình bày một phân tích toàn diện, bao quát và sâu sắc về tư duy giải quyết bài toán trong MLOps thông qua các hàm toán học, quá trình mô hình hóa các thành phần hệ thống thành các biến số trạng thái, và cách biểu diễn logic vận hành khắt khe thông qua các mã giả (pseudocode) chuẩn mực. Cấu trúc của báo cáo đi từ bài toán lập lịch luồng (pipeline scheduling) cơ bản ở tầng cơ sở hạ tầng, quản lý dữ liệu động tại hệ thống biên (Edge MLOps), các phương pháp toán học đo lường khoảng cách phân phối để tái sử dụng mô hình, cho đến việc tối ưu hóa hiệu quả kinh tế và tối đa hóa khả năng phục hồi của hệ thống y tế dựa trên học siêu tham số (meta-learning).

1. Tối Ưu Hóa Cấp Phát Tài Nguyên và Định Tuyến Khung Ống (MLOps Pipeline Scheduling & Task

Placement)

Trong các môi trường điện toán không đồng nhất (heterogeneous computing environments) điển hình của các trung tâm dữ liệu đám mây lớn, việc ánh xạ từng tác vụ (task) trong một ML pipeline vào đúng tài nguyên máy tính là một bài toán định tuyến NP-khó (NP-hard problem).⁷ Một ML pipeline chuẩn mực bao gồm một chuỗi các tác vụ đại diện cho các giai đoạn khác nhau trong vòng đời mô hình: từ tiền xử lý dữ liệu (data preprocessing), trích xuất đặc trưng, huấn luyện mô hình (model training), cho đến đánh giá và triển khai. Mỗi giai đoạn mang một đặc điểm tiêu thụ tài nguyên hoàn toàn khác biệt; ví dụ, tiền xử lý dữ liệu thường đòi hỏi băng thông và bộ nhớ RAM khổng lồ, trong khi huấn luyện mô hình học sâu lại đòi hỏi năng lực tính toán song song của GPU hoặc TPU.⁷ Để khai thác tối đa tính không đồng nhất này, các kỹ sư MLOps buộc phải xây dựng các bộ lập lịch (schedulers) vượt qua giới hạn của bộ lập lịch mặc định trong Kubernetes.

1.1. Mô Hình Toán Học Của Độ Phức Tạp Thuật Toán và Ràng Buộc Bộ Nhớ

Hệ thống lập lịch tối ưu hóa giải quyết bài toán định tuyến này thông qua một chiến lược hai giai đoạn được xác định rõ ràng: Lập lịch khung ống (Pipeline Scheduling) và Cấp phát tác vụ (Task Placement).⁷

Trong giai đoạn đầu tiên, để xác định thứ tự ưu tiên xử lý của các pipeline gửi đến hệ thống, MLOps hiện đại áp dụng thuật toán Ngắn Nhất Làm Trước (Shortest Job First - SJF). Thuật toán này đòi hỏi hệ thống phải có khả năng ước lượng chính xác khối lượng công việc thông qua độ phức tạp tính toán lý thuyết. Mặc dù các thư viện học máy thực tế (như TensorFlow hay PyTorch) đã được tối ưu hóa ở mức phần cứng, MLOps vẫn sử dụng số lượng phép toán lý thuyết (theoretical floating-point operations) làm hàm xấp xỉ (approximation function) để đo lường tải trọng huấn luyện, qua đó sắp xếp các luồng ưu tiên.⁷

Đối với giai đoạn thứ hai liên quan đến các tác vụ tiền xử lý, quyết định cấp phát tài nguyên phụ thuộc hoàn toàn vào khả năng của nút (node) tính toán để xử lý khối lượng dữ liệu đầu vào mà không gặp lỗi tràn bộ nhớ (out-of-memory). Ràng buộc cứng này được mô hình hóa toán học thông qua bất phương trình tài nguyên như sau ⁷:

$$M_{\text{available}}(n_i) > M_{\text{data}}(t_j) + M_{\text{overhead}}$$

Trong phương trình trên, $M_{\text{available}}(n_i)$ đại diện cho dung lượng bộ nhớ khả dụng theo thời gian thực trên nút thứ i thuộc cụm máy chủ. Tham số $M_{\text{data}}(t_j)$ là kích thước dữ liệu đầu vào dự kiến của tác vụ tiền xử lý thứ j . Cuối cùng, tham số M_{overhead} là biên độ bộ nhớ dự phòng tính được cấu hình để chứa các cấu trúc dữ liệu trung gian, các bản sao vùng nhớ và chi phí do quá trình thu gom rác (garbage collection) của ngôn ngữ lập trình sinh ra. Việc giải bài toán tối ưu hóa tổng thể một cách chính xác tuyệt đối sẽ mất rất nhiều thời gian, đi ngược lại

với yêu cầu thực thi thời gian thực của MLOps. Do đó, hệ thống ứng dụng một phương pháp giải heuristic dựa trên ma trận quy tắc ánh xạ (model-to-group mapping) kết hợp với việc cập nhật trạng thái tải trọng động của các nút.⁷

1.2. Định Nghĩa Biến Trạng Thái và Khung Logic Mã Giả Cấp Phát

Để hiện thực hóa phương pháp tiếp cận heuristic này một cách khả lập trình, quá trình cấp phát tác vụ cho các pipeline (khung ống) được biểu diễn qua các tập biến trạng thái cốt lõi. Bảng dưới đây định nghĩa cụ thể các biến số cấu thành không gian trạng thái của thuật toán ⁷:

Ký Hiệu Toán Học	Phân Loại Biến	Định Nghĩa Khái Niệm Trong Hệ Thống MLOps
P	Tập hợp (Set)	Danh sách các luồng (pipelines) chuẩn bị được thực thi, đã được sắp xếp giảm dần độ ưu tiên dựa trên ước lượng thuật toán SJF.
N	Tập hợp (Set)	Không gian các nút (nodes) tài nguyên có sẵn trong cụm máy chủ điện toán phân tán.
M	Ánh xạ (Mapping)	Quy tắc định tuyến từ sự kết hợp của (loại mô hình, loại tác vụ) sang các nhóm nút được cấu hình kiến trúc phần cứng tương ứng.
A	Cấu trúc dữ liệu (Map)	Ma trận kết quả lưu trữ cấu hình phân bổ cuối cùng của từng tác vụ vào các nút tương ứng.
L	Véc-tơ trạng thái	Véc-tơ theo dõi khối lượng tải hiện tại của từng nút, đo bằng tần suất các tác vụ đang được xử lý đồng thời.

<i>candidates</i>	Tập hợp con (Subset)	Không gian tìm kiếm thu hẹp chứa danh sách các nút đủ điều kiện (đáp ứng ràng buộc bộ nhớ) cho một tác vụ t cụ thể.
-------------------	----------------------	---

Dựa trên không gian trạng thái này, logic vận hành được cô đọng lại trong mã giả (Algorithm 1) mô tả thuật toán đặt tác vụ cơ sở (Pipeline Task Placement Base Algorithm). Phân tích mã giả này cho thấy tư duy giải quyết vấn đề linh hoạt thông qua việc lọc tập hợp và ra quyết định ưu tiên ⁷:

Algorithm 1: Thuật Toán Cấp Phát Tác Vụ MLOps Heuristic

Input:

P: Danh sách các pipeline đã sắp xếp bằng thuật toán SJF

N: Danh sách các nút điện toán trong cụm (cluster nodes)

M: Bản đồ quy tắc ánh xạ (model_type, task_type) -> node_groups

Output:

A: Bản đồ phân bổ tác vụ vào nút cho mỗi pipeline $p \in P$

1: $A \leftarrow \{\}$

2: $L \leftarrow \{\}$ ▷ Khởi tạo tải trọng của mỗi nút về 0

3: $p \leftarrow \text{null}$ ▷ Biến con trỏ cho pipeline hiện tại

4: $t \leftarrow \text{null}$ ▷ Biến con trỏ cho tác vụ hiện tại

5: $\text{candidates} \leftarrow \{\}$ ▷ Bộ nhớ đệm lưu trữ nút ứng cử viên

6: $\text{groups} \leftarrow \{\}$ ▷ Nhóm nút ràng buộc phần cứng từ bản đồ M

7: $\text{nodes} \leftarrow \{\}$ ▷ Các nút đã phân bổ trước đó cho pipeline p

8: $\text{common} \leftarrow \{\}$ ▷ Tập giao của 'nodes' và 'groups'

9: $\text{node_selected} \leftarrow \text{null}$

10:

11: for each p in P do

12: $A[p] \leftarrow \{\}$

13: for each t in p do

14: if t.type == "preprocessing" then

15: ▷ Lọc các nút thỏa mãn bất phương trình bộ nhớ

16: $\text{candidates} \leftarrow \text{filter_by_sufficient_memory}(N, t.\text{data})$

17: ▷ Tiêu chí chọn: nút có tải trọng (L) thấp nhất

18: $\text{node_selected} \leftarrow \text{least_loaded_node}(\text{candidates}, L)$

19: else

20: ▷ Đối với huấn luyện mô hình, truy xuất nhóm nút có phần cứng chuyên dụng

21: $\text{groups} \leftarrow \text{get_groups_from_mapping}(M, t.\text{model_type}, t.\text{type})$

22: $\text{candidates} \leftarrow \text{filter_nodes_by_groups}(N, \text{groups})$

23: $\text{candidates} \leftarrow \text{filter_by_sufficient_memory}(\text{candidates}, t.\text{data})$

```

24:
25: ▷ Ưu tiên tái sử dụng nút đã nạp dữ liệu cho pipeline p (locality)
26: nodes ← get_nodes_already_assigned_to(A[p])
27: common ← intersection(candidates, nodes)
28:
29: if length(common) > 0 then
30: node_selected ← least_loaded_node(common, L)
31: else if length(candidates) > 0 then
32: node_selected ← least_loaded_node(candidates, L)
33: else
34: ▷ Cơ chế chống bế tắc (deadlock/starvation prevention)
35: node_selected ← assign_fallback_node(N, L)
36: end if
37: end if
38:
39: ▷ Ghi nhận trạng thái hệ thống sau khi cấp phát
40: A[p][t] ← node_selected
41: L[node_selected] ← L[node_selected] + 1 ▷ Tính tiền tải trọng
42: end for
43: end for
44:
45: return A

```

Thuật toán trên thể hiện một quá trình lọc phân tầng tinh vi. Bắt đầu từ dòng 11 đến 43, thuật toán lặp qua từng pipeline theo thứ tự ưu tiên SJF. Điểm sáng của logic này nằm ở khối điều kiện (dòng 14-37). Nếu tác vụ là tiên xử lý (đòi hỏi I/O và RAM), thuật toán ưu tiên đánh giá dung lượng bộ nhớ. Ngược lại, nếu là tác vụ huấn luyện (đòi hỏi GPU), thuật toán (dòng 25-28) tìm cách tận dụng nguyên lý địa phương dữ liệu (data locality) bằng cách kiểm tra phép giao (intersection) giữa các nút có phần cứng phù hợp và các nút đã thực hiện tác vụ tiên xử lý trước đó. Nếu tìm thấy tập giao này, hệ thống sẽ chọn nút ít tải nhất trong đó, giúp triệt tiêu hoàn toàn độ trễ truyền tải dữ liệu qua mạng nội bộ. Việc tích hợp logic định tuyến heuristic này giúp hệ thống MLOps tự động hóa việc đưa ra quyết định ở mức cơ sở hạ tầng, giảm trung bình 68% tổng thời gian thực thi (execution time) và hơn 80% thời gian chờ đợi đối với các ML pipeline phức tạp so với trình lập lịch Kube-scheduler tiêu chuẩn.⁷

Đối với các môi trường vĩ mô kết nối Đám mây và vạn vật (Cloud-IoT), bài toán thậm chí còn phức tạp hơn, đòi hỏi hệ thống phải đáp ứng độ trễ (latency), tiêu thụ năng lượng và chất lượng dịch vụ (QoS). Trong bối cảnh này, các kiến trúc MLOps tiên tiến mở rộng thuật toán cấp phát cơ bản bằng cách kết hợp Tối ưu hóa Đàn kiến (Ant Colony Optimization - ACO) chuyên đảm nhiệm không gian tìm kiếm tổ hợp toàn cục, kết hợp với các tác tử Học tăng cường sâu (Deep RL) để thích ứng liên tục với biến động khối lượng công việc.⁸

2. Quản Lý Dữ Liệu Động Tại Biên và Tối Ưu Hóa Bộ Đặc Trưng (Dynamic Data Management in Edge

MLOps)

Bên cạnh tối ưu hóa tài nguyên tính toán đám mây, một nhánh phát triển mang tính cách mạng khác trong MLOps là khả năng quản lý vòng đời dữ liệu liên tục tại các thiết bị phần cứng hạn chế, đặc biệt là trong các ứng dụng điện toán biên (Edge MLOps) như hệ thống gợi ý chăm sóc sức khỏe dựa trên thiết bị đeo IoT (Wearable IoT Data).⁹ Vấn đề căn cốt ở đây là hiệu ứng trôi dạt mô hình (model drift hay model decay) – sự suy giảm chất lượng dự đoán dần dần theo thời gian khi dữ liệu chuỗi thời gian sinh ra liên tục làm thay đổi đặc tính thống kê so với tập huấn luyện gốc.⁹ Sự biến thiên không ngừng của dữ liệu cảm biến sinh học yêu cầu mô hình phải liên tục thích ứng, nhưng băng thông truyền tải và năng lượng điện toán tại biên không cho phép tái huấn luyện toàn bộ dữ liệu lịch sử.⁹

Thay vì huấn luyện lại mô hình một cách mù quáng tốn kém, giới nghiên cứu đã đề xuất một phương pháp tiếp cận hướng dữ liệu (Data-Driven Optimization) được phát triển dưới dạng quy trình Quản lý Dữ liệu Động (Dynamic Data Management - DDM).⁹ Mục tiêu của DDM là tối ưu hóa bộ nhớ và thời gian tính toán tại biên thông qua nén dữ liệu và tinh giản đặc trưng (feature selection), nhằm bảo toàn tín hiệu cốt lõi cần thiết cho các mô hình gợi ý trong khi loại bỏ độ trễ rác.¹³

2.1. Cấu Trúc Khung Tính Toán Trích Xuất Dữ Liệu và Học Lọc Tín Hiệu

Quá trình thu gọn và lựa chọn đặc trưng trong DDM không phụ thuộc vào các ngưỡng cố định mà được vận hành bởi một chuỗi các mô hình toán học lồng nhau. Đầu tiên, dữ liệu chuỗi thời gian (time-series) thô từ các phiên hoạt động (sessions) của người dùng được nén thành các véc-tơ có độ dài cố định dựa trên sự trích xuất các khoảnh khắc thống kê (statistical moments) cơ bản¹³:

$$\mathbf{S}_{user} = [\mu(X), \sigma(X), \min(X), \max(X)]$$

Để giảm số chiều (dimensionality reduction) của không gian trạng thái này mà vẫn giữ nguyên phương sai chính (primary variance), hệ thống sử dụng thuật toán Phân tích Thành phần Chính (Principal Component Analysis - PCA). PCA tìm ra một hệ tọa độ trực giao mới sao cho trục hoành biểu diễn phương sai lớn nhất. Tiếp theo, việc đánh giá tính dư thừa của dữ liệu quá khứ so với dữ liệu mới được tính toán qua phương pháp Độ tương đồng Cosine (Cosine Similarity) bằng việc đối chiếu hai véc-tơ không gian latent. Nếu sự phân mảnh cosine giữa hai chuỗi hành vi quá thấp (vượt qua một SimilarityThreshold), dữ liệu lịch sử sẽ bị cắt tỉa khỏi bộ nhớ để giải phóng tài nguyên.¹³

Sau khi nén dữ liệu theo chiều dọc (số lượng mẫu), hệ thống tiếp tục nén theo chiều ngang (số lượng đặc trưng). Quyết định duy trì một đặc trưng m phụ thuộc vào Bộ điều khiển Perceptron đa lớp (MLP Controller). Bộ điều khiển này nhận vào véc-tơ tiềm ẩn z đại diện cho cấu trúc dữ

liệu tổng hợp và xuất ra một phân phối trọng số tính toán a_{nm} quyết định mức độ quan trọng của đặc trưng đó. Mô hình tối ưu hóa này áp dụng kiến trúc mạng nơ-ron hai lớp (two-layer MLP) có hàm kích hoạt phi tuyến ReLU và cơ chế Regularization bằng Dropout (với tỷ lệ loại bỏ ngẫu nhiên $rate = 0.2$) nhằm tăng tính khái quát của bộ lọc trước các nhiễu loạn ¹³:

$$a_{nm} = \text{ReLU}(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 z + \mathbf{b}_1) + \mathbf{b}_2)$$

Trong phương trình mạng nơ-ron này, $\mathbf{W}_1, \mathbf{W}_2$ là ma trận trọng số và $\mathbf{b}_1, \mathbf{b}_2$ là các véc-tơ độ lệch (bias). Điểm đặc biệt của quy trình huấn luyện bộ kiểm soát này là việc sử dụng hàm mục tiêu tối ưu hóa hai cấp (bi-level optimization). Thay vì chỉ cập nhật dựa trên hàm mất mát thông thường, sai số trên tập dữ liệu kiểm định (validation loss) được dùng trực tiếp làm tín hiệu giám sát cấp cao (supervision signal) để điều hướng quá trình tính toán gradient cho các giá trị trọng số $\mathbf{W}$, đảm bảo rằng việc loại bỏ một đặc trưng không làm phương hại đến tính chính xác tổng quát của hệ thống gọi ý.¹³

2.2. Biểu Diễn Mã Giả Của DDM Workflow (Quy Trình Quản Lý Dữ Liệu Động)

Với các phép tính phức tạp trên, quá trình quản lý dữ liệu động được bọc gói trong một luồng công việc tự động (pipeline) dành cho MLOps tại biên. Bảng dưới đây thể hiện các thành phần biến chính tham gia vào quy trình xử lý ¹³:

Khái Niệm	Ký Hiệu Biến Trong DDM Workflow	Mục Đích Luồng Công Việc
Dữ liệu Lịch sử	OldData	Tập dữ liệu IoT cũ đang lưu trữ trên thiết bị, chứa các mẫu hành vi cốt lõi đã được hấp thụ.
Dữ liệu Mới	NewData	Tập dữ liệu vừa được thu thập trực tiếp từ cảm biến thiết bị đeo tay theo thời gian thực.
Mẫu Dữ liệu Nén	UserPatternData	Phiên bản cô đọng cực hạn của dữ liệu lịch sử, sau khi đã áp dụng PCA và lọc Cosine Similarity.

Trọng số Tín hiệu	FeatureImportances	Véc-tơ trọng số a_{nm} xuất ra từ mạng MLP, quyết định giữ hay bỏ các cột đặc trưng.
-------------------	--------------------	--

Mã giả tổng quát (Algorithm 2) cho DDM Workflow thể hiện tư duy kết hợp chặt chẽ giữa lọc thông tin (information filtering) và tiền xử lý (preprocessing) nhằm tạo ra một tập huấn luyện kích thước nhỏ nhất nhưng mang lại hiệu năng cao nhất ⁹:

Algorithm 2: Dynamic Data Management (DDM) Workflow cho Recommender Systems

Input:

Old_data (Tập CSV chứa dữ liệu quá khứ)

New_data (Tập CSV chứa dữ liệu IoT mới thu thập)

Output:

Optimized_Training_Set (Tập dữ liệu tối ưu với kích thước tối thiểu để huấn luyện lại)

1: function dynamic_data_management(Old_data, New_data):

2: ▷ Nạp dữ liệu thô từ hệ thống tập tin nội bộ

3: OldData, NewData ← read_csv(Old_data), read_csv(New_data)

4:

5: ▷ Tiền xử lý dữ liệu mới để đồng nhất định dạng và dải giá trị

6: NewData_preprocessed ← preprocess(NewData)

7:

8: ▷ Khởi chạy phân tích kép (Dual-Analysis) để nén dữ liệu theo hai chiều

9: UserPatternData, FeatureWeights ← analyze_data(OldData, NewData_preprocessed)

10:

11: ▷ -----

12: ▷ BÊN TRONG HÀM analyze_data THỰC THI HAI TIỀN TRÌNH TOÁN HỌC:

13: ▷ Tiến trình 1: Nén dữ liệu theo chiều dọc (Mẫu quan sát)

14: ▷ - Trích xuất S_user (mean, std, min, max) từ OldData.

15: ▷ - Chạy PCA để giảm số chiều không gian mẫu.

16: ▷ - Áp dụng Cosine Similarity loại bỏ các session dư thừa so với NewData.

17: ▷ - Kết quả xuất ra UserPatternData.

18: ▷ Tiến trình 2: Khởi tạo mô hình định giá đặc trưng (MLP Controller)

19: ▷ - Truyền véc-tơ tiềm ẩn z (tổng hợp UserPatternData & NewData) vào MLP.

20: ▷ - Chạy lan truyền xuôi qua 2 lớp ReLU để tính toán FeatureWeights (a_{nm}).

21: ▷ -----

22:

23: ▷ Dựa vào mức trọng số, chọn lọc các đặc trưng có ý nghĩa nhất

24: FeatureImportance ← select_important_features(FeatureWeights)

25:

26: ▷ Hợp nhất và định tuyến dữ liệu để tạo ra tập huấn luyện cuối cùng


```
27: Optimized_Training_Set ← package_data(UserPatternData, NewData_preprocessed,  
FeatureImportance)
```

```
28:
```

```
29: return Optimized_Training_Set
```

```
30: end function
```

Hệ quả trực tiếp của luồng thao tác logic này là sự sụt giảm đáng kể tài nguyên dành cho việc retraining hệ thống. Nhờ thuật toán nén thông minh, các nghiên cứu thực nghiệm xác nhận DDM Workflow làm giảm kích thước dữ liệu cần thiết theo hệ số thu gọn (Reduction Rate) cấu hình sẵn, nhưng ngược lại, gia tăng chỉ số hiệu suất xếp hạng NDCG (Normalized Discounted Cumulative Gain) cho bộ máy gợi ý y tế nhờ loại bỏ thành công các tín hiệu nhiễu từ thiết bị cảm biến.¹³ Kiến trúc MLOps tại biên theo đó chuyển đổi từ một quy trình tiêu thụ tài nguyên khổng lồ thành một hệ thống hoạt động bền vững cho thiết bị cấu hình thấp.

3. Nhận Diện Trôi Dạt Dữ Liệu và Chiến Lược Tái Sử Dụng Mô Hình Dựa Trên Khoảng Cách Phân Phối (Data Drift Detection & SimReuse)

Một nghịch lý tồn tại dai dẳng trong quản trị vòng đời ML truyền thống (MLOps 1.0) là việc các kỹ sư thường ứng phó với hiệu ứng trôi dạt dữ liệu (data drift) và trôi dạt khái niệm (concept drift) bằng cách cài đặt lịch tái huấn luyện tự động định kỳ (retraining schedules) — ví dụ như lập lịch chạy vào rạng sáng mỗi ngày hoặc mỗi cuối tuần.¹⁷ Chiến lược "chữa cháy" mù quáng này gây ra chi phí điện toán khổng lồ và lãng phí tín chỉ đám mây không đáng có, đặc biệt đối với dữ liệu chuỗi thời gian (time-series) có đặc tính phân phối lặp lại theo chu kỳ (seasonality), như biến động giá cả, nhiệt độ thời tiết, hay mật độ giao thông.¹²

Để giải quyết bài toán toán học về sự lãng phí này, các nhà nghiên cứu đã thiết lập một kiến trúc MLOps tiên tiến gọi là SimReuse (Similarity-Based Model Reuse).¹⁹ Nền tảng tư duy ở đây cực kỳ trực quan nhưng cần công cụ định lượng phức tạp: Nếu phân phối dữ liệu tại cửa sổ thời gian (time window) hiện tại trên môi trường sản xuất hoàn toàn tương đồng với phân phối dữ liệu tại một cửa sổ thời gian trong quá khứ, hệ thống có thể tái sử dụng trực tiếp mô hình tĩnh (artifact) đã được huấn luyện tốt trong quá khứ thay vì lãng phí tài nguyên để khởi chạy quy trình huấn luyện một mô hình hoàn toàn mới.¹⁹

3.1. Các Độ Đo Toán Học Xác Định Tính Tương Đồng Phân Phối

Việc nhận diện sự dịch chuyển hay lặp lại của các phân phối không thể chỉ dựa trên trực giác phân tích đồ thị hình hộp (box plots) mà phải được lượng hóa thông qua hai phương pháp đo lường khoảng cách thống kê cực kỳ nghiêm ngặt¹²:

1. Khoảng cách Wasserstein (Wasserstein Distance - WD / Earth Mover's Distance):

Trong bối cảnh trôi dạt dữ liệu, Khoảng cách Wasserstein tính toán chi phí vận chuyển khối

lượng tối thiểu (minimum effort) để biến đổi hàm mật độ phân phối xác suất P của tập dữ liệu

quá khứ thành phân phối xác suất Q của tập dữ liệu mới trên một không gian metric nhất định.

Lợi thế vượt trội của WD so với các tiêu chuẩn thông thường (như phân kỳ Kullback-Leibler) là khả năng nắm bắt được hình dạng hình học tổng thể của phân phối, hoạt động tốt ngay cả khi các phần hỗ trợ (supports) của hai phân phối không giao nhau. Biểu diễn toán học của Khoảng

cách Wasserstein bậc p cho hai phân phối xác suất liên tục P và Q là:

$$W_p(P, Q) = \left(\inf_{\gamma \in \Gamma(P, Q)} \int_{M \times M} d(x, y)^p d\gamma(x, y) \right)^{1/p}$$

Trong đó, $\Gamma(P, Q)$ đại diện cho tập hợp tất cả các phân phối đồng thời (joint distributions) khả thi có các phân phối biên (marginals) lần lượt tương ứng với P và Q , trong khi $d(x, y)$ là hàm khoảng cách cơ sở (thường là khoảng cách Euclidean). Giá trị WD càng nhỏ chứng tỏ hai chuỗi thời gian càng có sự tương đồng phân phối lớn.¹²

2. Khoảng cách Biến thiên Toàn phần (Total Variation Distance - TVD):

TVD đóng vai trò bổ trợ, đo lường sự chênh lệch tuyệt đối lớn nhất của xác suất mà hai phân phối gán cho cùng một biến cố thuộc không gian mẫu $\mathcal{F}$. Về mặt toán học, đây là một tiêu chuẩn chặt chẽ để phát hiện các bất thường đột ngột về biên độ:

$$\delta_{TV}(P, Q) = \sup_{A \in \mathcal{F}} |P(A) - Q(A)| = \frac{1}{2} \int |p(x) - q(x)| dx$$

Bên cạnh các phép đo phân phối trên toàn cục, đối với các nền tảng học máy trực tuyến (online stream learning), hệ thống phải nhận diện trôi dạt khái niệm qua luồng dữ liệu liên tục. Một phương pháp như thuật toán EStream (Ensemble Learning Approach) sử dụng cơ chế ngưỡng

tin cậy dự đoán (Threshold Confidence - TC) và Bộ đếm thể hiện (Instance Counter - IC_C). Phương pháp này tính toán tỷ lệ tín nhiệm liên tục cho các phân loại viên (classifiers). Nếu một

điểm dữ liệu không thỏa mãn bất phương trình $PC \geq TC$ (với PC là Predicted Confidence của lớp dự đoán), hệ thống cảnh báo rằng cấu trúc logic tiềm ẩn đã bị dịch chuyển.¹⁷

3.2. Cấu Trúc Thuật Toán Tái Sử Dụng Mô Hình (SimReuse Pseudocode)

Trong khuôn khổ kiến trúc SimReuse, khi WD và TVD được kiểm định nằm dưới một ngưỡng siêu tham số ϵ được thiết lập từ trước, hệ thống kết luận rằng trôi dạt khái niệm đang diễn ra theo tính chất chu kỳ.¹² Thay vì gửi lệnh kích hoạt luồng CI/CD để huấn luyện lại, một lệnh tái cấp phát mô hình gốc (artifact promotion) sẽ được gọi.²² Căn cứ theo tài liệu nghiên cứu của Majidi và cộng sự¹², mã giả được diễn dịch qua các thao tác duyệt (looping) trên các khối dữ

liệu:

Algorithm 3: Thuật Toán Tái Sử Dụng Mô Hình Dựa Trên Khoảng Cách (SimReuse)

Input:

Dataset (Dữ liệu chuỗi thời gian, định dạng Pandas Dataframe)

W (Kích thước cửa sổ thời gian - window size)

Target_column (Cột biến mục tiêu cần dự báo)

Similarity_Dictionary (Từ điển {Key: Index cửa sổ hiện tại, Value: Index các cửa sổ quá khứ tương đồng},

được xây dựng dựa trên ma trận khoảng cách WD và TVD)

Forecasting_Approach (Chuỗi định danh thuật toán, ví dụ: "LSTM", "RF", "XGB")

Output:

Ghi log dự đoán hiệu suất và danh sách các mô hình được triển khai qua thời gian

1: saved_models $\leftarrow$ Danh sách lưu trữ các đối tượng mô hình (artifacts) đã từng huấn luyện

2:

3: $\triangleright$ Duyệt qua toàn bộ tập dữ liệu bằng cơ chế cửa sổ trượt (sliding window)

4: for each data_segment_index from 0 to floor(length(Dataset)/W) do

5: $\triangleright$ Trích xuất ma trận dữ liệu con tại cửa sổ hiện thời

6: current_window $\leftarrow$ extract_window(Dataset, data_segment_index, W)

7:

8: $\triangleright$ Kiểm tra ma trận khoảng cách: Có phân phối tương tự trong quá khứ không?

9: if data_segment_index in keys(Similarity_Dictionary) then

10: similar_past_index $\leftarrow$ Similarity_Dictionary[data_segment_index]

11:

12: $\triangleright$ Bỏ qua Retraining. Truy xuất trực tiếp mô hình phù hợp từ bộ nhớ (S3 Bucket/Registry)

13: active_model $\leftarrow$ get_model(saved_models, similar_past_index)

14:

15: $\triangleright$ Đánh giá mô hình cũ trên dữ liệu mới để thẩm định tính khả thi

16: performance $\leftarrow$ evaluate(active_model, current_window)

17: log_metrics(performance, status="Reused")

18: else

19: $\triangleright$ Tình huống 1: Dữ liệu khởi tạo (Bootstrap).

20: $\triangleright$ Tình huống 2: Bắt gặp Concept Drift hoàn toàn mới, chưa từng thấy trong quá khứ.

21: active_model $\leftarrow$ train_new_model(current_window, Forecasting_Approach)

22: saved_models.append(active_model)

23:

24: performance $\leftarrow$ evaluate(active_model, current_window)

25: log_metrics(performance, status="Retrained")

26: end if

27:

28: $\triangleright$ Đẩy Artifact lên môi trường Production MLOps (với GitOps-ish rollouts)

29: deploy_to_production(active_model)

30: end for

Phân tích dòng mã từ 9 đến 26 cho thấy sức mạnh cốt lõi của SimReuse. Nếu không có bộ từ

điểm kiểm tra tính tương đồng (được xây dựng trước bởi WD/TVD), vòng lặp này sẽ luôn rẽ nhánh vào hàm `train_new_model` tại dòng 21 cho mỗi đoạn dữ liệu mới, gây thảm họa tài nguyên đám mây. Nhờ lồng ghép tư duy nhận diện chu kỳ trực tiếp vào quá trình đánh giá bằng các metric toán học phân phối, hệ thống MLOps giảm thiểu số lần huấn luyện lặp thừa thãi xuống hơn 15 lần về mặt thời gian và chi phí tính toán (đạt mức tiết kiệm chi tiêu phần cứng lên tới 87.5%), trong khi vẫn duy trì hiệu suất học máy ngang ngửa hoặc vượt trội so với baseline (mô hình dự báo tĩnh).¹⁸ Môi trường phi sản xuất (NonProd-MLOps) xử lý các mô hình thử nghiệm, trong khi môi trường sản xuất (Prod-MLOps) thực thi các tác vụ tái sử dụng này dưới chế tài kiểm soát truy cập nghiêm ngặt của chuẩn công nghiệp.²⁴

4. Mô Hình Hóa Quyết Định MLOps Qua Quá Trình Quyết Định Markov và Học Tăng Cường (MDP & RL in MLOps)

Sự phức tạp của việc lựa chọn khi nào nên tái huấn luyện, tái sử dụng, hay bỏ qua một hiện tượng trôi dạt nhỏ không thể luôn được hard-code bằng các ngưỡng tĩnh. Hướng tới thể hệ tương lai của các chiến lược đào tạo liên tục (Continuous Training Policies), MLOps đang tích hợp các nguyên lý của Quá trình Quyết định Markov (Markov Decision Processes - MDP) và Học Tăng Cường (Reinforcement Learning - RL) để tự động hóa hoàn toàn các thao tác ra quyết định ở tầng quản lý vòng đời hệ thống (Lifecycle Management).⁶

4.1. Công Thức Hóa Không Gian Quyết Định MLOps theo MDP

Trong bối cảnh MLOps, động lực hệ thống (system dynamics) được biểu diễn bởi một bộ 4 tham số (4-tuple) cốt lõi (S, A, P_a, R_a) , trong đó không gian thời gian được chia thành các bước rời rạc (discrete time-steps).⁶ Sự tương ứng giữa lý thuyết điều khiển và thực tiễn MLOps được thể hiện rõ ràng như sau:

- **Không gian trạng thái S (State):** Bao gồm tất cả các chỉ số đo lường hệ thống tại một thời điểm, chẳng hạn như mức độ phân kỳ dữ liệu hiện tại, tải trọng CPU/GPU của cụm máy chủ, ngân sách điện toán còn lại, và độ trễ phản hồi dự đoán.
- **Không gian hành động A (Action):** Các quyết định mà tác tử quản lý MLOps (Agent) có thể đưa ra. Các hành động điển hình bao gồm: Kích hoạt Retraining (a_1), Tái sử dụng mô hình gốc (a_2), Hủy bỏ luồng công việc (a_3), hoặc Triển khai mô hình dự phòng nhẹ hơn (a_4).
- **Hàm chuyển trạng thái $P_a(s'|s, a)$:** Xác suất hệ thống chuyển sang trạng thái s' nếu tác tử thực thi hành động a tại trạng thái s .

- **Hàm phần thưởng $R_a(s, a)$ (Reward Function):** Tín hiệu toán học định hướng học máy. Đây là yếu tố sống còn để cân bằng giữa độ chính xác của mô hình và chi phí hệ thống.⁶

Để tối đa hóa tổng phần thưởng kỳ vọng (expected utility) với hệ số chiết khấu $\gamma \in [0, 1]$, hệ thống sử dụng thuật toán Q-Learning hoặc Policy Gradient để tìm ra chính sách tối ưu $\pi(s)$:

$$V^{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R_t | s_0 = s \right]$$

4.2. Cầu Dao Tự Động Qua Điểm Số Hiệu Quả Tái Huấn Luyện (RES)

Điểm nguy hiểm nhất khi tích hợp RL vào tự động hóa hạ tầng là hiện tượng "hack phần thưởng" (Reward Hacking).²⁵ Tác tử RL trong môi trường đám mây có thể học được chính sách luôn luôn ra lệnh "Retrain" với dữ liệu khổng lồ để đạt được độ chính xác tuyệt đối mà không đếm xỉa đến ngân sách GPU của doanh nghiệp. Trong thực tiễn ngành công nghiệp, "cái bẫy độ chính xác" (accuracy trap) thường xảy ra khi các nhà khoa học dữ liệu nỗ lực cải thiện 1% độ chính xác của mô hình nhưng lại tiêu tốn tài nguyên phần cứng lớn gấp 10 lần mức bình thường.²⁸

Để giải quyết cấu trúc khiếm khuyết này và thiết lập hàm phần thưởng hợp lý cho RL, MLOps 2.0 đưa quản trị tài chính trực tiếp vào quy trình kỹ thuật thông qua một công cụ toán học nghiêm ngặt: Điểm số Hiệu quả Tái huấn luyện (Retraining-Efficiency Score - RES).²⁸ Hệ thống hoạt động như một "cầu dao tự động" (programmatic circuit breaker). Phương trình tính toán đánh đổi thời gian thực giữa mức tăng lợi ích biên (marginal gain in accuracy) và chi phí điện toán phát sinh (compute cost) để đạt được nó:

$$RES = \frac{\Delta \text{Accuracy}}{\Delta \text{Compute Cost}}$$

Tác tử RL sẽ nhận một hình phạt (negative reward) nặng nề nếu điểm RES rơi xuống dưới một ngưỡng λ do nghiệp vụ kinh doanh quy định.²⁸

- Nếu mô hình là một thuật toán giao dịch tần suất cao (high-frequency trading), 1% độ chính xác tăng thêm có thể mang về hàng triệu đô-la lợi nhuận. Khi đó, hàm phần thưởng được cấu hình với ngưỡng λ rất thấp để ưu tiên chất lượng tối đa bất chấp chi phí điện toán.
- Nếu mô hình chỉ là một hệ thống chatbot trả lời các ticket IT nội bộ, sự chênh lệch 1% gần như vô nghĩa đối với trải nghiệm người dùng cuối. Ngưỡng λ lúc này được đẩy lên cực cao để RL agent thiên về chiến lược bảo toàn năng lượng và tối ưu hóa ngân sách đám mây.²⁸

Thuật toán cầu dao bảo vệ ngân sách (Circuit Breaker) này được tích hợp thẳng vào bước

Continuous Training (CT) trong luồng CI/CD (như GitHub Actions hoặc GitLab CI).²⁸ Nó ngăn chặn hệ thống triển khai các bản cập nhật vô nghĩa, qua đó ràng buộc lý thuyết điều khiển tự động với hiệu quả kinh tế vĩ mô.

5. MLOps Xanh và Tối Ưu Hóa Năng Lượng Qua Cơ Chế Xếp Tầng Mô Hình (Green AI Dynamic Model Cascading)

Trong bối cảnh năng lượng tiêu thụ của trung tâm dữ liệu đang trở thành mối bận tâm toàn cầu, MLOps không chỉ tập trung vào hiệu suất mà còn phải giải bài toán "AI Xanh" (Green AI). Thay vì dùng một mô hình học sâu khổng lồ duy nhất cho mọi điểm dữ liệu (vốn tiêu tốn cực kỳ nhiều năng lượng phân tích), chiến lược xếp tầng mô hình động (Dynamic Model Cascading) đề xuất việc sử dụng một hệ sinh thái gồm nhiều mô hình có độ phức tạp và mức tiêu thụ năng lượng tăng dần.³⁰

5.1. Khung Toán Học Định Tuyến Năng Lượng

Giả sử ta có một tập dữ liệu phân loại được gán nhãn $D = (x_i, y_i)_{i=1}^n$ với $x_i \in X$ là không gian đặc trưng và $y_i \in Y$ là không gian nhãn. Hệ thống MLOps triển khai một tập hợp gồm $k \geq 2$ mô hình phân loại $M_1, M_2, \dots, M_k$. Các mô hình này được sắp xếp một cách nghiêm ngặt theo thứ tự mức độ tiêu thụ năng lượng (và thường là độ phức tạp cấu trúc tính toán) tăng dần.³⁰

Cốt lõi toán học của phương pháp này nằm ở việc xác định một họ các hàm độ tin cậy dự đoán (prediction confidence functions) phụ thuộc vào đầu ra của từng mô hình M_i :

$$\alpha_i(x) : X \times Y \rightarrow$$

Đi kèm với hàm đo lường này là một siêu tham số dung sai độ tin cậy (confidence tolerance

parameter) ký hiệu là ϵ . Biến số ϵ đại diện cho mức độ sẵn sàng đánh đổi giữa độ chính xác và lượng năng lượng tiết kiệm được. Dung sai càng lớn, hệ thống càng dễ chấp nhận kết quả từ mô hình yếu hơn để giảm thiểu tài nguyên tính toán.

5.2. Thuật Toán Mã Giả Xếp Tầng (Dynamic Model Cascading Algorithm)

Tại thời điểm suy diễn (inference time) trên môi trường sản xuất, cơ chế định tuyến Green AI

hoạt động như một phễu lọc.³⁰ Khi một điểm dữ liệu đầu vào x được đưa tới API, MLOps pipeline không chuyển thẳng nó cho mô hình LLM lớn nhất mà bắt đầu kích hoạt mạng lưới mô

hình theo quy trình thuật toán đệ quy vòng lặp như sau:

Algorithm 4: Cơ Chế Suy Diễn Xếp Tầng Green AI (Dynamic Model Cascading)

Input:

x (Điểm dữ liệu đầu vào cần suy diễn)

$M_list = [M_1, M_2, \dots, M_k]$ (Danh sách mô hình đã sắp xếp tăng dần theo mức tiêu thụ năng lượng)

$\alpha_functions = [\alpha_1, \alpha_2, \dots, \alpha_k]$ (Danh sách hàm tính độ tin cậy tương ứng)

ϵ (Ngưỡng dung sai độ tin cậy, $0 \leq \epsilon \leq 1$)

Output:

y_pred (Nhãn phân loại cuối cùng cho dữ liệu x)

1: function green_ai_cascade_inference($x, M_list, \alpha_functions, \epsilon$):

2: ▷ Lặp qua chuỗi các mô hình từ tiết kiệm năng lượng nhất đến tốn kém nhất

3: for $i = 1$ to k do

4: ▷ Kích hoạt mô hình M_i và lấy kết quả phân loại dự đoán

5: $y_pred_candidate \leftarrow M_list[i].predict(x)$

6:

7: ▷ Tính toán điểm tin cậy nội tại của dự đoán (VD: Giá trị Softmax lớn nhất)

8: $confidence_score \leftarrow \alpha_functions[i].calculate_confidence(x, y_pred_candidate)$

9:

10: ▷ Kiểm tra bất phương trình chấp nhận kết quả

11: if $confidence_score \geq (1 - \epsilon)$ then

12: ▷ Độ tin cậy đủ lớn, hệ thống ngắt mạch và chấp nhận dự đoán ngay lập tức,

13: ▷ giải phóng hoàn toàn các mô hình cấp cao hơn ($M_{\{i+1\}} \dots M_k$) khỏi việc chạy điện toán.

14: return $y_pred_candidate$

15: end if

16:

17: ▷ Nếu độ tin cậy thấp, mô hình M_i đã thất bại trong việc giải quyết ca khó này,

18: ▷ vòng lặp sẽ đẩy x sang mô hình $M_{\{i+1\}}$ có độ phức tạp cao hơn.

19: end for

20:

21: ▷ Fallback: Trả về kết quả của mô hình M_k (tốn kém nhất và chính xác nhất)

22: ▷ nếu tất cả các mô hình sơ cấp đều không thỏa mãn ngưỡng dung sai.

23: return $y_pred_candidate$

24: end function

Phân tích logic của đoạn mã chỉ ra sự thanh lịch của phương pháp tối ưu này. Nếu một bài toán

quá đơn giản, mô hình M_1 (chẳng hạn một mô hình Linear Regression cực nhẹ) có thể đạt

điểm tự tin 95%. Nếu $\epsilon = 0.1$, điều kiện $0.95 \geq (1 - 0.1)$ được thỏa mãn ở dòng 11. Hệ thống ngắt lệnh ngay lập tức mà không cần đánh thức một mạng Deep Neural Network hàng tỷ

tham số nằm ở M_k . Sự kết hợp tinh vi giữa toán học đánh giá rủi ro (confidence scoring) và cấu trúc luồng điều khiển (control flow) đã giúp kiến trúc AI tiết kiệm vô số chu kỳ điện năng mà không làm xói mòn chất lượng dịch vụ tổng thể, duy trì một MLOps bền vững với môi trường.³¹

6. Tối Ưu Hóa Khả Năng Phục Hồi Dựa Trên Học Siêu Tham Số (Resilience-Aware MLOps)

Nếu quản lý tài nguyên và phát hiện trôi dạt dữ liệu giải quyết vấn đề hiệu năng và chi phí, thì trong các ứng dụng đặc thù và trọng yếu khẩn khe - như hệ thống chẩn đoán y tế dựa trên AI (AI-based medical diagnostic systems) - tính bền bỉ và khả năng tự phục hồi (Resilience) trước nhiễu loạn mới là yếu tố tối thượng quyết định sự thành bại.² Các rủi ro như tấn công đối kháng (adversarial attacks), lỗi tiêm nhiễm (fault injection) phần cứng máy quét MRI, và sự chệch hướng phân phối ngoài dữ liệu huấn luyện đòi hỏi hệ thống phải có cơ chế phòng vệ nội sinh.³⁴ Một khung MLOps Nhận thức Khả năng Phục hồi (Resilience-aware MLOps) được thiết kế đặc biệt nhằm nâng cao sự tin cậy (trustworthiness) thông qua việc tự động hóa quá trình phòng vệ.³⁴

6.1. Công Thức Hóa Chỉ Số Phục Hồi Tích Phân và Đạo Hàm Siêu Gradient

Sự phục hồi của một hệ thống học máy không phải là sự cứng nhắc (robustness) đơn thuần mà là khả năng hệ thống duy trì được năng lực sau khi hấp thụ một lượng nhiễu loạn khổng lồ, và sở hữu đặc tính "thoái hóa từ tốn" (graceful degradation) thay vì sụp đổ hoàn toàn.² Kiến trúc

này đưa vào toán học một bộ chỉ báo phục hồi tích phân (integral resilience indicator) R_{τ_i}

dùng để đo lường khả năng tồn tại của hệ thống trong lần thực thi nhiễu τ_i trên các tham số mô hình ω và bộ dữ liệu kiểm tra $D_{\tau_i}^{val}$.

Hệ thống được vận hành bởi một toán tử thích nghi U , kết hợp việc tạo ra nhiễu chủ động và quá trình thích ứng cục bộ trong T bước, nhằm ánh xạ trạng thái hiện tại ϕ sang trạng thái mới của ϕ . Phương trình độ phục hồi được định nghĩa chặt chẽ như sau ³⁴:

$$R_{\tau_i} = \frac{1}{P_0 T} \sum_{t=1}^T P_{\tau_i}(\theta, \omega, \phi_t, W_t, D_{\tau_i}^{val})$$

Trong phương trình trên:

- P_0 là mức hiệu suất cơ sở tuyệt đối của hệ thống trước khi gặp bất kỳ sự cố nào.
- P_{τ_i} là hàm đo lường hiệu suất (metric, ví dụ: F1-Score hoặc Accuracy) ở trạng thái hiện tại của tham số mô hình tại vòng lặp t .

- θ, ω, ϕ_t đại diện cho không gian tham số cục bộ và vĩ mô của hệ thống (như ma trận weights, biases của adapter và frozen blocks).
- W_t biểu thị cấu trúc các nhiễu loạn môi trường sinh ra (ví dụ: mất một vùng pixel lớn do cảm biến X-Quang hỏng hóc).

Để giải quyết bài toán tối ưu hàm R_{τ_i} , hệ thống triển khai phương pháp Học Siêu Tham Số Không Phụ Thuộc Mô Hình (Model-Agnostic Meta-Learning - MAML) sử dụng các Chiến lược Tiến hóa (Evolution Strategies - ES).³⁴ Tại vòng lặp bên ngoài (outer loop) của kiến trúc tối ưu hóa, ước tính siêu độ dốc (meta-gradient estimation) được thực hiện qua một phiên bản làm mịn Gaussian của hàm mục tiêu thay vì tính đạo hàm Backpropagation thông thường (do hàm mục tiêu có thể không khả vi liên tục tại điểm nhiễu loạn).

Mục tiêu cốt lõi là tối ưu hóa tập tham số khởi tạo Ξ sao cho mô hình có khả năng thích nghi nhanh nhất với bất kỳ dạng nhiễu loạn ngẫu nhiên nào. Phép tính đạo hàm được định nghĩa qua công thức kỳ vọng³⁴:

$$\nabla_{\Xi} \mathbb{E}_{\mathbf{g} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} F(\mathbf{X}_i + \sigma \mathbf{g}) = \frac{1}{2\sigma} \mathbb{E}_{\mathbf{g} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\frac{\partial F(\mathbf{X}_i + \sigma \mathbf{g})}{\partial \mathbf{g}} \right]$$

Với $\mathbf{g}$ là véc-tơ biến thiên (perturbation vector) được sinh ra từ phân phối chuẩn đa chiều $\mathcal{N}(\mathbf{0}, \mathbf{I})$ tại mỗi bước meta-optimization iteration, và σ đại diện cho hệ số làm mịn phân tán của Gaussian.³⁴

6.2. Mã Giả Học Siêu Tham Số Tiến Hóa Cho MLOps Y Tế

Logic toán học trừu tượng trên được thể hiện một cách mạch lạc qua thuật toán điều khiển giả định (Pseudocode of MAML with ES for AI-system resilience optimization).² Đây là trái tim của quy trình rèn luyện khả năng phục hồi tự động, tách biệt lập trình viên y tế khỏi gánh nặng thiết kế cơ chế chống lỗi thủ công.³³

Algorithm 5: Học Siêu Tham Số MAML Với Chiến Lược Tiến Hóa Cho Khả Năng Phục Hồi
Input:

Distribution of tasks/disturbances $p(\mathbf{T})$ (Phân phối các kịch bản nhiễu loạn mô phỏng)

Step size hyperparameters α, β (Tốc độ học cho vòng lặp trong và vòng lặp ngoài)

Smoothing factor σ (Hệ số làm mịn của Gaussian)

Output:

Meta-optimized parameters Ξ (Bộ tham số siêu tối ưu cho các thành phần adapter phòng vệ)

1: Khởi tạo ngẫu nhiên ma trận các tham số vĩ mô Ξ

2: while không hội tụ (not done) do

3: Lấy mẫu một batch (nhóm) các kịch bản nhiễu loạn $\{\tau_i\}$ từ phân phối $p(\mathbf{T})$

4:

5: Sinh ra một véc-tơ nhiễu loạn ngẫu nhiên $\mathbf{g}$ từ phân phối chuẩn cho vòng lặp ngoài

6: $\mathbf{g} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

7:

8: for each kịch bản nhiễu loạn τ_i do

```

9: ▷ Thiết lập 2 bản sao của mô hình với biến thiên thuận (+) và biến thiên nghịch (-)
10:  $\Xi_{\text{plus}} \leftarrow \Xi + \sigma * g$ 
11:  $\Xi_{\text{minus}} \leftarrow \Xi - \sigma * g$ 
12:
13: ▷ Vòng lặp thích ứng nội bộ (Inner Loop Adaptation - Thực thi Toán tử U)
14: for  $t = 1$  to  $T$  do
15: Đánh giá gradient  $\nabla L_{\text{ti}}(f_{\Xi_{\text{plus}}})$  trên dữ liệu bị tiêm nhiễu
16: Cập nhật tạm thời các tham số nội bộ bằng SGD (với tốc độ học  $\alpha$ )
17: end for
18:
19: ▷ Sau  $T$  bước rèn luyện, đánh giá Chỉ số phục hồi tích phân  $R(\Xi)$  trên validation set
20: Tính  $R_{\text{plus}} = R(\Xi_{\text{plus}})$  sử dụng công thức tích phân  $R_{\text{ti}}$  (thực thi trên dữ liệu ngoài mẫu)
21: Tính  $R_{\text{minus}} = R(\Xi_{\text{minus}})$  sử dụng công thức tích phân  $R_{\text{ti}}$ 
22: end for
23:
24: ▷ Cập nhật siêu tham số tại vòng lặp ngoài (Outer Loop Update) sử dụng Gradient tiến hóa
25:  $\text{Meta-gradient} \leftarrow (1 / 2\sigma) * g$ 
26:  $\Xi \leftarrow \Xi + \beta * \text{Meta-gradient}$ 
27: end while
28:
29: return  $\Xi$ 

```

Phân tích mã giả (Algorithm 5) cho thấy sự lồng ghép giữa hai vòng lặp. Vòng lặp trong (Inner Loop, dòng 13-17) ép mạng nơ-ron phải vật lộn với những hình ảnh y khoa đã bị làm hỏng (tiêm

nhiều) trong T bước hữu hạn, mô phỏng quá trình thích nghi nhanh. Vòng lặp ngoài (Outer Loop, dòng 24-26) không tối ưu hóa để giải quyết trực tiếp một loại nhiễu cụ thể nào, mà dùng

Gradient tiến hóa để dịch chuyển tọa độ khởi tạo véc-tơ Ξ tới một điểm cực trị nằm trong không gian tham số, nơi mà từ điểm đó, mạng nơ-ron có thể hội tụ nhanh chóng để tự chữa lành trước bất cứ bất thường nào sinh ra từ thiết bị phần cứng thực tế.

Bằng cách liên tục đẩy mô hình vào trạng thái phải tự phục hồi từ các giá trị ngẫu nhiên g , mô-đun adapter (thành phần tinh chỉnh) học được phản xạ đáp ứng với những trường hợp lệch phân phối ngoài miền (out-of-distribution data). Trong cấu trúc MLOps này, nhà nghiên cứu AI y tế được giải phóng hoàn toàn khỏi nhiệm vụ viết code chống nhiễu loạn phần cứng thủ công để tập trung triệt để vào khả năng nhận diện hình ảnh bệnh lý học.³⁴ Kết quả từ thực nghiệm đối chứng trên các tập dữ liệu thực tế (như MedMNIST) chứng minh sự phục hồi vượt trội về hiệu suất phân loại — tăng tới 24% độ phục hồi cho mạng tích chập (CNN) và 19.7% cho mô hình Visual Transformers (ViT) dưới điều kiện tiêm lỗi (fault injections) so với các pipeline MLOps truyền thống.³⁴

Phân Tích Đa Tầng và Tác Động Cấp Số Nhân Của MLOps Định Lượng

Việc hệ thống hóa toàn bộ vòng đời học máy thành một mạng lưới các hàm toán học chặt chẽ và thuật toán tối ưu tạo ra những tác động lan truyền đáng kinh ngạc, vượt ra ngoài quy mô kiểm soát kỹ thuật cục bộ, tái định nghĩa lại tính chất của ngành kỹ nghệ phần mềm thông minh. Thứ nhất, tính minh bạch định lượng (quantitative transparency) đã thiết lập một tiêu chuẩn quản trị rủi ro mới. Việc mã hóa trôi dạt khái niệm thành Khoảng cách Wasserstein (WD) hay đo

lượng độ bền phục hồi qua biến tích phân R_{τ_i} đã chuyển trạng thái MLOps từ các thử nghiệm mang tính chất phòng thí nghiệm của kỹ sư (DevOps scripts) sang một hệ thống quản trị tự động hóa hoàn toàn có thể tái lập (reproducible).³ Khi mọi quyết định—from việc chọn Nút máy chủ xử lý tác vụ (qua phân phối SJF heuristic), loại bỏ tập dữ liệu lịch sử (qua Cosine

Similarity), tới việc ngắt mạch triển khai mô hình (qua ngưỡng λ của RES)—đều được truy vết về một hệ bất phương trình chặt chẽ, thì việc tuân thủ các quy chuẩn khắt khe về công nghệ như luật AI Act của châu Âu hay các yêu cầu thanh tra kiểm toán nội bộ trở thành tính năng cốt lõi (built-in feature) chứ không còn là một bộ quy tắc áp đặt.³

Thứ hai, tác động tương hỗ giữa các thuật toán tạo ra một hệ sinh thái cộng sinh hoàn hảo. Các mã giả và khung toán học này không tồn tại riêng lẻ mà sở hữu độ tương kết cấu trúc sâu sắc. Để đạt được việc tái sử dụng mô hình tối đa (SimReuse) tại các hệ thống biên²⁰, MLOps phải tận dụng quy trình tinh giản luồng dữ liệu liên tục (DDM Workflow) để làm sạch và ép chuỗi tín hiệu thô vào không gian latent ổn định bằng mạng MLP.¹³ Dữ liệu sau khi nén càng gọn nhẹ thì việc tính toán chi phí vận chuyển phân phối (Wasserstein Distance) càng tiết kiệm chu kỳ vì xử lý, từ đó gia tăng độ giãn của ngưỡng "cắt dao" bảo vệ ngân sách (RES) trong khâu triển khai vòng lặp CI/CD. Ngược lại, nếu thiết bị thu thập bị ảnh hưởng bởi lỗi nhiễu cảm biến IoT phần cứng nghiêm trọng², thuật toán tối ưu hóa siêu tham số MAML với ES³⁴ sẽ tự động kích hoạt khung kháng lỗi nội bộ ngay trước khi truyền tải lên Cloud, chặn đứng hội chứng "rác vào, rác ra" (garbage in, garbage out) ngay từ tuyến đầu của hệ thống thu thập thông tin. Quá trình kiểm soát và tương hỗ này tạo ra một vòng lặp kín (closed-loop feedback) đảm bảo hệ thống duy trì được tính tin cậy tuyệt đối và liên tục tinh chỉnh sự hoàn hảo về toán học theo thời gian thực.³

Nguồn trích dẫn

1. [2501.14802] DNN-Powered MLOps Pipeline Optimization for Large Language Models: A Framework for Automated Deployment and Resource Management - arXiv, truy cập vào tháng 5 29, 2026, <https://arxiv.org/abs/2501.14802>
2. Resilience-aware MLOps for AI-based medical diagnostic system - ResearchGate, truy cập vào tháng 5 29, 2026, https://www.researchgate.net/publication/379751661_Resilience-aware_MLOps_for_AI-based_medical_diagnostic_system
3. MLOps 2.0: A Reference Architecture for CI/CD with Always-On Data Quality Gates, truy cập vào tháng 5 29, 2026, <https://www.preprints.org/manuscript/202511.2095>
4. MLOps: Continuous delivery and automation pipelines in machine learning | Cloud Architecture Center, truy cập vào tháng 5 29, 2026,

<https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>

5. "It Was a Magical Box": Understanding Practitioner Workflows and Needs in Optimization, truy cập vào tháng 5 29, 2026, <https://arxiv.org/html/2509.16402v1>
6. Self-Adaptive Machine Learning-based Systems - Carnegie Mellon University, truy cập vào tháng 5 29, 2026, <http://reports-archive.adm.cs.cmu.edu/anon/s3d2024/CMU-S3D-24-112.pdf>
7. Optimising ML Pipeline Execution via Smart Task Placement - MDPI, truy cập vào tháng 5 29, 2026, <https://www.mdpi.com/2079-9292/14/13/2555>
8. Daily Papers - Hugging Face, truy cập vào tháng 5 29, 2026, <https://huggingface.co/papers?q=adaptive%20data%20scheduling>
9. Data-Driven Optimization of Healthcare Recommender System Retraining Pipelines in MLOps with Wearable IoT Data - MDPI, truy cập vào tháng 5 29, 2026, <https://www.mdpi.com/1424-8220/25/20/6369>
10. [PDF] Data-Driven Optimization of Healthcare Recommender, truy cập vào tháng 5 29, 2026, <https://www.semanticscholar.org/paper/Data-Driven-Optimization-of-Healthcare-Recommender-Park-Mun/fe49f9dbe5852b41853ebfa628e7f66199362bc6>
11. Catch My Drift? A Beginner's Guide to ML Model Drift | by Saundarya Subramaniam, truy cập vào tháng 5 29, 2026, <https://medium.com/@saundaryasubramaniam/catch-my-drift-a-beginners-guide-to-ml-model-drift-ac3c0127235d>
12. An Efficient Model Maintenance Approach for MLOps - arXiv, truy cập vào tháng 5 29, 2026, <https://arxiv.org/pdf/2412.04657?>
13. Data-Driven Optimization of Healthcare Recommender System ..., truy cập vào tháng 5 29, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12568114/>
14. AdaFS: Adaptive Feature Selection in Deep Recommender System | Request PDF - ResearchGate, truy cập vào tháng 5 29, 2026, https://www.researchgate.net/publication/362690251_AdaFS_Adaptive_Feature_Selection_in_Deep_Recommender_System
15. Data-Driven Optimization of Healthcare Recommender System Retraining Pipelines in MLOps with Wearable IoT Data - ResearchGate, truy cập vào tháng 5 29, 2026, https://www.researchgate.net/publication/396602490_Data-Driven_Optimization_of_Healthcare_Recommender_System_Retraining_Pipelines_in_MLOps_with_Wearable_IoT_Data
16. Data-Driven Optimization of Healthcare Recommender System Retraining Pipelines in MLOps with Wearable IoT Data - PubMed, truy cập vào tháng 5 29, 2026, <https://pubmed.ncbi.nlm.nih.gov/41157422/>
17. (PDF) ElStream: An Ensemble Learning Approach for Concept Drift Detection in Dynamic Social Big Data Stream Learning - ResearchGate, truy cập vào tháng 5 29, 2026, https://www.researchgate.net/publication/351175011_ElStream_An_Ensemble_Learning_Approach_for_Concept_Drift_Detection_in_Dynamic_Social_Big_Data_Stream_Learning

18. An Efficient Model Maintenance Approach for MLOps | Request PDF - ResearchGate, truy cập vào tháng 5 29, 2026, https://www.researchgate.net/publication/386555243_An_Efficient_Model_Maintenance_Approach_for_MLOps?_tp=eyJjb250ZXh0lp7lnBhZ2UiOiJzY2llbnRpZmljQ29udHJpYnV0aW9ucyIsInByZXZpb3VzUGFnZSI6bnVsbH19
19. [2412.04657] An Efficient Model Maintenance Approach for MLOps - arXiv, truy cập vào tháng 5 29, 2026, <https://arxiv.org/abs/2412.04657>
20. ForoughMajidi1995/Maintenance - GitHub, truy cập vào tháng 5 29, 2026, <https://github.com/ForoughMajidi1995/Maintenance>
21. forough majidi | Mila, truy cập vào tháng 5 29, 2026, <https://mila.quebec/en/directory/forough-majidi>
22. MLOps-Driven Software Engineering: Designing Feedback- Loop Architectures for Intelligent Applications - ResearchGate, truy cập vào tháng 5 29, 2026, https://www.researchgate.net/publication/396984372_MLOps-Driven_Software_Engineering_Designing_Feedback-Loop_Architectures_for_Intelligent_Applications
23. An efficient model maintenance approach for MLOps | Request PDF - ResearchGate, truy cập vào tháng 5 29, 2026, https://www.researchgate.net/publication/397682218_An_efficient_model_maintenance_approach_for_MLOps
24. Analytics Ingestion Service - Politecnico di Torino, truy cập vào tháng 5 29, 2026, <https://webthesis.biblio.polito.it/secure/36347/1/tesi.pdf>
25. Integrating Reinforcement Learning into Agentic AI Systems | by Samin Chandeepea (Adomic) | Medium, truy cập vào tháng 5 29, 2026, <https://medium.com/@saminchandeepea/integrating-reinforcement-learning-into-agentic-ai-systems-7b45d3d6883f>
26. ML & MLOps Development | Machine Learning Operations Platform - Xenoss, truy cập vào tháng 5 29, 2026, <https://xenoss.io/capabilities/ml-mlops>
27. RLOps: Development Life-cycle of Reinforcement Learning Aided Open RAN - arXiv, truy cập vào tháng 5 29, 2026, <https://arxiv.org/pdf/2111.06978>
28. The AI Accuracy Trap: Why MLOps Needs a Financial Circuit Breaker - VKTR, truy cập vào tháng 5 29, 2026, <https://www.vktr.com/ai-technology/the-ai-accuracy-trap/>
29. MLOps Explained: Automate & Scale Machine Learning Workflows, truy cập vào tháng 5 29, 2026, <https://kestra.io/resources/ai/what-is-mlops>
30. Choosing to Be Green: Advancing Green AI via Dynamic Model Selection, truy cập vào tháng 5 29, 2026, <https://ceur-ws.org/Vol-4165/short2.pdf>
31. Choosing to Be Green: Advancing Green AI via Dynamic Model Selection - arXiv, truy cập vào tháng 5 29, 2026, <https://arxiv.org/pdf/2509.19996>
32. Advancing Green AI via Dynamic Model Selection - IRIS, truy cập vào tháng 5 29, 2026, <https://iris.universitaediroma.it/retrieve/b01acec9-82b3-4e4b-8630-134975f5ebf95/short2.pdf>
33. Resilience-aware MLOps for AI-based medical diagnostic system - PubMed, truy cập vào tháng 5 29, 2026, <https://pubmed.ncbi.nlm.nih.gov/38601490/>

34. Resilience-aware MLOps for AI-based medical diagnostic system ..., truy cập vào tháng 5 29, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11004236/>
35. Resilience-aware MLOps for AI-based medical diagnostic system - Semantic Scholar, truy cập vào tháng 5 29, 2026, <https://www.semanticscholar.org/paper/Resilience-aware-MLOps-for-AI-based-medical-system-Moskalenko-Kharchenko/1a95d35e1fe74725a6f8d9d123d77c02a768d7db>
36. Resilience-aware MLOps for AI-based medical diagnostic system - Frontiers, truy cập vào tháng 5 29, 2026, <https://www.frontiersin.org/journals/public-health/articles/10.3389/fpubh.2024.1342937/full>
37. Maturity Framework for Operationalizing Machine Learning Applications in Health Care: Scoping Review - Journal of Medical Internet Research, truy cập vào tháng 5 29, 2026, <https://www.jmir.org/2025/1/e66559>
38. Industrial MLOps: a systematic review of architectures and implementation challenges, truy cập vào tháng 5 29, 2026, <https://www.tandfonline.com/doi/full/10.1080/21693277.2026.2658878>